

# Economic Intelligence & Financial Markets Platform

An end-to-end, containerized ELT data platform that ingests macroeconomic indicators and cryptocurrency market data, models it into a strict PostgreSQL Gold layer, and serves it through interactive operational dashboards — with automated CI/CD validation on every change.

Python 3.10

PostgreSQL

Docker & Docker Compose

Streamlit + Plotly

GitHub Actions CI/CD

## 01 Executive Summary

This project is a production-style data platform, not a notebook demo: it extracts data from three independent live APIs, enforces a relational schema on write, orchestrates every step with dependency-aware sequencing, and ships as versioned, containerized infrastructure validated by CI on every push.

The platform continuously tracks two domains that move markets: Brazilian macroeconomic indicators (inflation, unemployment, and the USD/BRL exchange rate) and the Bitcoin market (historical price and trading volume). Both domains land in the same PostgreSQL "Gold" layer — a single, query-ready source of truth — and are exposed through a Streamlit dashboard for interactive analysis.

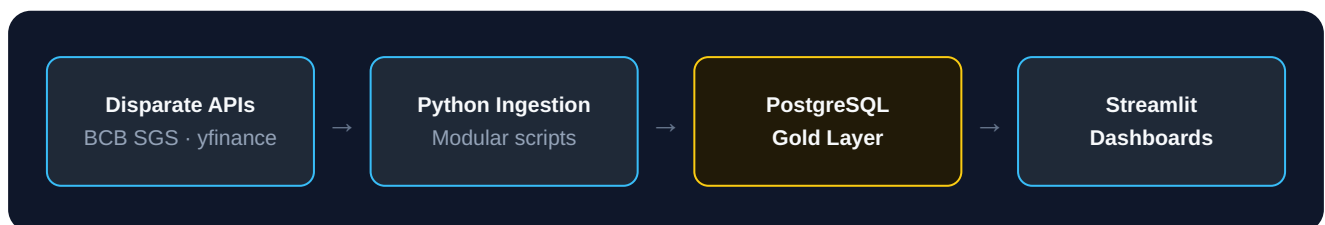
|                   |                           |                             |                           |                        |
|-------------------|---------------------------|-----------------------------|---------------------------|------------------------|
| <b>4</b>          | <b>3,300+</b>             | <b>2,400+</b>               | <b>3</b>                  | <b>100%</b>            |
| LIVE DATA SOURCES | MACRO RECORDS<br>INGESTED | BITCOIN RECORDS<br>INGESTED | CONTAINERIZED<br>SERVICES | CI-VALIDATED<br>PUSHES |

### Why this project exists

Most portfolio projects stop at "I can pull an API and make a chart." This one is built to demonstrate the layer above that: how to take that same data and turn it into infrastructure — something a team could actually deploy, monitor, and extend. Every architectural choice below was made with that bar in mind, and the reasoning behind each one is documented explicitly rather than left implicit in the code.

## 02 Architecture & Pipeline Flow

The platform is deliberately modular. Extraction, storage, orchestration, and presentation are isolated into separate layers, so each one can be tested, replaced, or scaled independently without touching the others.



### Layer responsibilities

- **Ingestion** — independent Python scripts per data domain (Bitcoin, macroeconomics), each responsible only for fetching, cleaning, and typing its own data.

- **Storage (Gold layer)** — a single PostgreSQL database enforcing schema-on-write, so every downstream consumer reads clean, typed, deduplicated data.
- **Orchestration** — a central script sequences ingestion steps, fails fast on error, and logs every stage for traceability.
- **Presentation** — a Streamlit application queries the Gold layer directly and renders interactive, filterable charts with Plotly.

## 03 Key Engineering Decisions

---

Below are the decisions that most shaped this platform — each one made as a deliberate trade-off, not a default.

### ELT instead of ETL

Raw JSON payloads are small enough to clean in-memory with **pandas** before loading — there's no need for a heavyweight transform-in-warehouse step at this data volume. Keeping transformation in Python keeps the pipeline simple, fast to iterate on, and easy to unit test.

### A single PostgreSQL "Gold" layer as the source of truth

Rather than exposing raw API responses to the dashboard, every ingestion script writes into the same relational database. This gives the presentation layer one consistent, typed, query-ready interface — and gives the whole platform ACID guarantees instead of ad-hoc file reads.

### Docker Compose with explicit health checks

The pipeline and dashboard containers declare `depends_on: condition: service_healthy` on Postgres. This removes a common race condition in multi-container setups — the application containers never attempt a connection before the database has actually finished initializing.

### One ingestion script per data domain

`bitcoin_ingestion.py` and `macro_ingestion.py` are fully independent. A failure or schema change in one cannot silently break the other, and each can be run, tested, or extended in isolation — a direct application of single-responsibility design to a data pipeline.

### A thin orchestrator with fail-fast logging

`orchestrator.py` runs each step as a subprocess, captures its output, and stops the whole pipeline immediately if any step fails — instead of silently continuing with partial data. Every step is timestamped and logged, so a failure is traceable to the exact script and error.

### Streamlit + Plotly for the presentation layer

Streamlit turns a pandas/SQL pipeline into an interactive internal tool without building and maintaining a separate frontend stack — the fastest path from "data in Postgres" to "chart a stakeholder can filter and explore."

## CI/CD with an ephemeral Postgres service

GitHub Actions spins up a disposable PostgreSQL container on every push and pull request to validate the platform's dataframe/schema contracts — catching structural regressions before they ever reach a deployed environment, with zero persistent test infrastructure to maintain.

## 04 Pipeline Walkthrough

This section walks through a full pipeline run end to end, with evidence captured directly from the running system.

### 1 Schema initialization

*Why: guarantees the Gold layer's tables exist and are correctly typed before any data is written — the pipeline never assumes the database is already set up.*

`database_init.py` connects to PostgreSQL and creates `gold_economic_indicators` and `gold_bitcoin_metrics` if they don't already exist.

### 2 Ingestion & orchestration

*Why: sequencing ingestion through a single orchestrator makes multi-source failures visible and traceable instead of silent.*

The orchestrator runs the Bitcoin and macroeconomic ingestion scripts in sequence, logging each stage. The excerpt below is captured directly from a live run:

```
PS C:\Users\Usuario\projeto_economia_dados\infrastructure> docker compose logs pipeline
economics_pipeline | [*] Initializing Gold layer schema and analytical tables...
economics_pipeline | [+] Gold layer tables successfully initialized.
economics_pipeline | 2026-09-08 18:38:12,486 - INFO - Starting complete data platform orchestration...
economics_pipeline | 2026-09-08 18:38:12,486 - INFO - [Database Initialization] Running script: database_init.py
economics_pipeline | 2026-09-08 18:38:12,822 - INFO - Success in [Database Initialization]:
economics_pipeline | [*] Initializing Gold layer schema and analytical tables...
economics_pipeline | [+] Gold layer tables successfully initialized.
economics_pipeline | 2026-09-08 18:38:12,822 - INFO - [Bitcoin Data Ingestion] Running script: analytics_models/bitcoin_ingestion.py
economics_pipeline | 2026-09-08 18:38:16,889 - INFO - Success in [Bitcoin Data Ingestion]:
economics_pipeline | [*] Downloading historical Bitcoin (BTC-USD) data...
economics_pipeline | [*] Inserting 2443 records into the gold_bitcoin_metrics table...
economics_pipeline | [+] Bitcoin ingestion completed successfully.
economics_pipeline | 2026-09-08 18:38:16,889 - INFO - [Macroeconomic Data Ingestion] Running script: analytics_models/macro_ingestion
economics_pipeline | .py
economics_pipeline | 2026-09-08 18:38:23,839 - INFO - Success in [Macroeconomic Data Ingestion]:
economics_pipeline | [*] Fetching IPCA from Central Bank of Brazil (BCB SGS Series 433)...
economics_pipeline | [+] Successfully loaded 139 records for IPCA.
economics_pipeline | [*] Fetching Unemployment Rate from Central Bank of Brazil (BCB SGS Series 24369)...
economics_pipeline | [+] Successfully loaded 139 records for Unemployment Rate.
economics_pipeline | [*] Fetching USD/BRL Exchange Rate via yfinance...
economics_pipeline | [+] Successfully loaded 3040 records for USD/BRL Exchange Rate.
economics_pipeline | [*] Inserting total of 3318 macroeconomic records into gold_economic_indicators...
economics_pipeline | [+] Macroeconomic multi-indicator ingestion completed successfully with 0 errors.
economics_pipeline | 2026-09-08 18:38:23,839 - INFO - Pipeline executed successfully from end to end.
PS C:\Users\Usuario\projeto_economia_dados\infrastructure>
```

*Live orchestration log — full pipeline run, end to end*

### 3 Query-ready Gold layer

*Why: proves the schema-on-write contract holds — data lands typed and query-ready, not as raw JSON blobs.*

Once loaded, the data is immediately queryable with plain SQL — no post-processing required downstream.

```
economics_gold=# SELECT date, indicator_name, value FROM gold_economic_indicators LIMIT 10;
```

| date       | indicator_name     | value |
|------------|--------------------|-------|
| 2015-01-01 | IPCA - Monthly (%) | 1.24  |
| 2015-02-01 | IPCA - Monthly (%) | 1.22  |
| 2015-03-01 | IPCA - Monthly (%) | 1.32  |
| 2015-04-01 | IPCA - Monthly (%) | 0.71  |
| 2015-05-01 | IPCA - Monthly (%) | 0.74  |
| 2015-06-01 | IPCA - Monthly (%) | 0.79  |
| 2015-07-01 | IPCA - Monthly (%) | 0.62  |
| 2015-08-01 | IPCA - Monthly (%) | 0.22  |
| 2015-09-01 | IPCA - Monthly (%) | 0.54  |
| 2015-10-01 | IPCA - Monthly (%) | 0.82  |

(10 rows)

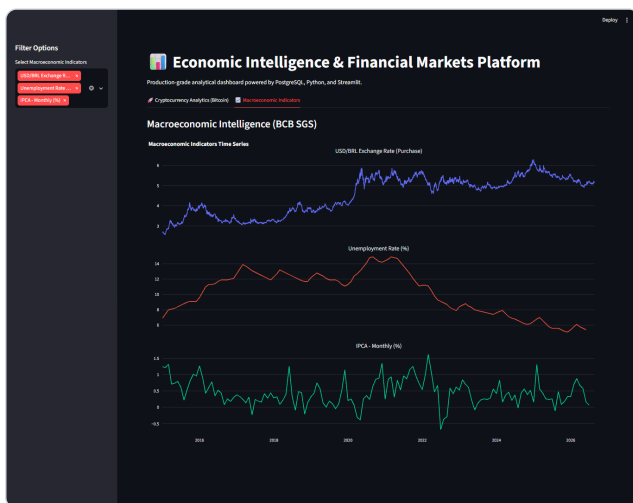
Direct SQL query against gold\_economic\_indicators

## 4 Interactive presentation

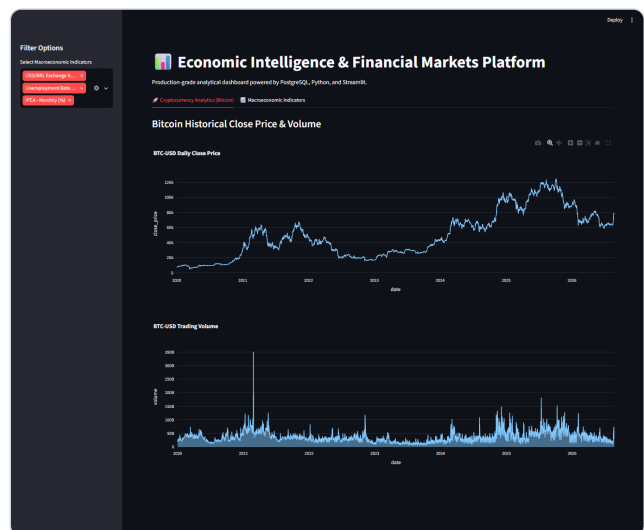
*Why: validates that the Gold layer is genuinely consumable — a real dashboard, filtering and rendering live data, not a static export.*

The Streamlit dashboard queries the Gold layer directly and renders both domains as independent, filterable views.

The dashboard renders each domain on its own tab, with independent filters:



Macroeconomic tab — USD/BRL, unemployment, IPCA

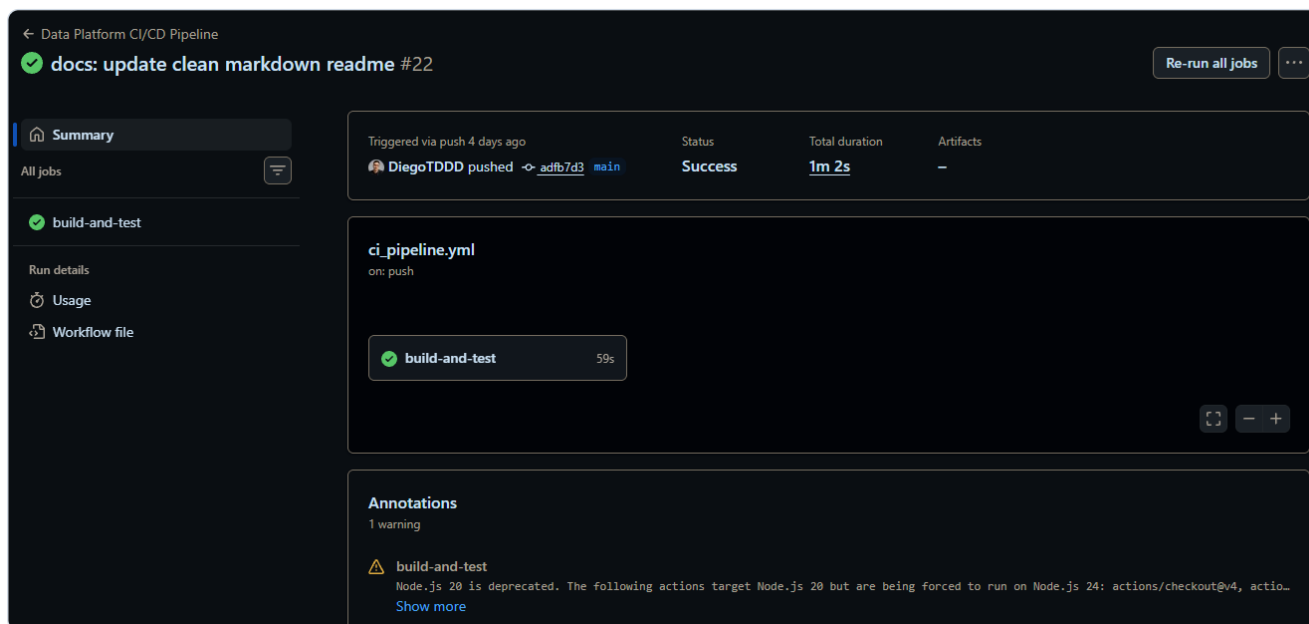


Cryptocurrency tab — BTC-USD price and volume

## 5 Continuous validation

*Why: catches structural regressions automatically, before a broken schema or contract ever reaches a deployed environment.*

Every push and pull request triggers GitHub Actions, which installs the exact pinned dependencies and runs the platform's test suite against a disposable PostgreSQL instance.



GitHub Actions — CI pipeline passing on the main branch

## 05 Technology Stack

| LAYER          | COMPONENT                                                    | PURPOSE                                                          |
|----------------|--------------------------------------------------------------|------------------------------------------------------------------|
| Ingestion      | Python ( <a href="#">requests</a> , <a href="#">pandas</a> ) | API data extraction, JSON parsing, and type enforcement.         |
| Storage        | <a href="#">PostgreSQL</a>                                   | ACID-compliant relational storage for clean time-series metrics. |
| Orchestration  | <a href="#">orchestrator.py</a>                              | Execution sequencing and fail-fast dependency handling.          |
| Infrastructure | <a href="#">Docker</a> & <a href="#">Docker Compose</a>      | Network isolation and environment-agnostic reproducibility.      |
| CI/CD          | <a href="#">GitHub Actions</a>                               | Automated dependency install and test run on every push.         |
| Presentation   | <a href="#">Streamlit</a> & <a href="#">Plotly</a>           | Interactive operations console and real-time visualization.      |

### Data sources

#### Banco Central do Brasil — SGS API

IPCA inflation index (Series 433) and Unemployment Rate (Series 24369), fetched live on every pipeline run.

#### yfinance API

USD/BRL exchange rate and historical Bitcoin (BTC-USD) close price and trading volume.

*Note: all data is extracted dynamically at runtime — no static datasets are hardcoded in the repository.*

## 06 Skills Demonstrated

---

### Data Pipeline Engineering

ELT design, orchestration, fail-fast error handling, idempotent schema-on-write.

### SQL & Relational Modeling

PostgreSQL schema design, typed columns, constraints, direct analytical querying.

### Containerization

Multi-service Docker Compose, health checks, dependency-aware startup ordering.

### CI/CD

GitHub Actions with ephemeral services, automated test validation on every push.

### API Integration

REST (BCB SGS) and third-party SDK (yfinance) consumption with error handling.

### Data Visualization

Streamlit + Plotly interactive, filterable, multi-panel dashboards.

### Python Engineering

Modular scripts, single-responsibility design, pandas data cleaning at scale.

### Version Control Hygiene

Clean git history, .gitignore discipline, environment-variable configuration.

## 07 Running the Platform

---

The entire platform is designed to start with a single command.

- `git clone https://github.com/DiegoTDDD/Economic-data-platform.git`
- `cd Economic-data-platform/infrastructure`
- `docker compose up -d --build`
- Dashboard available at `http://localhost:8501`

Full source code, commit history, and CI runs are public.

[github.com/DiegoTDDD/Economic-data-platform](https://github.com/DiegoTDDD/Economic-data-platform)